

Lecture 11

CC 483

Introduction to Mobile Cloud Computing: Architecture and applications of MCC, Code partitioning, Code offloading and VM migration techniques.

Mobile Cloud Computing

Mobile Cloud Computing (MCC) is a technology where **mobile devices (smartphones, tablets)** use **cloud computing resources** (servers, storage, processing power) over the internet to run applications and store data.

Mobile Cloud Computing **allows mobile phones to perform heavy tasks by using powerful cloud servers** instead of relying only on their own hardware.

Why Do We Need Mobile Cloud Computing?

Mobile devices have **limitations**:

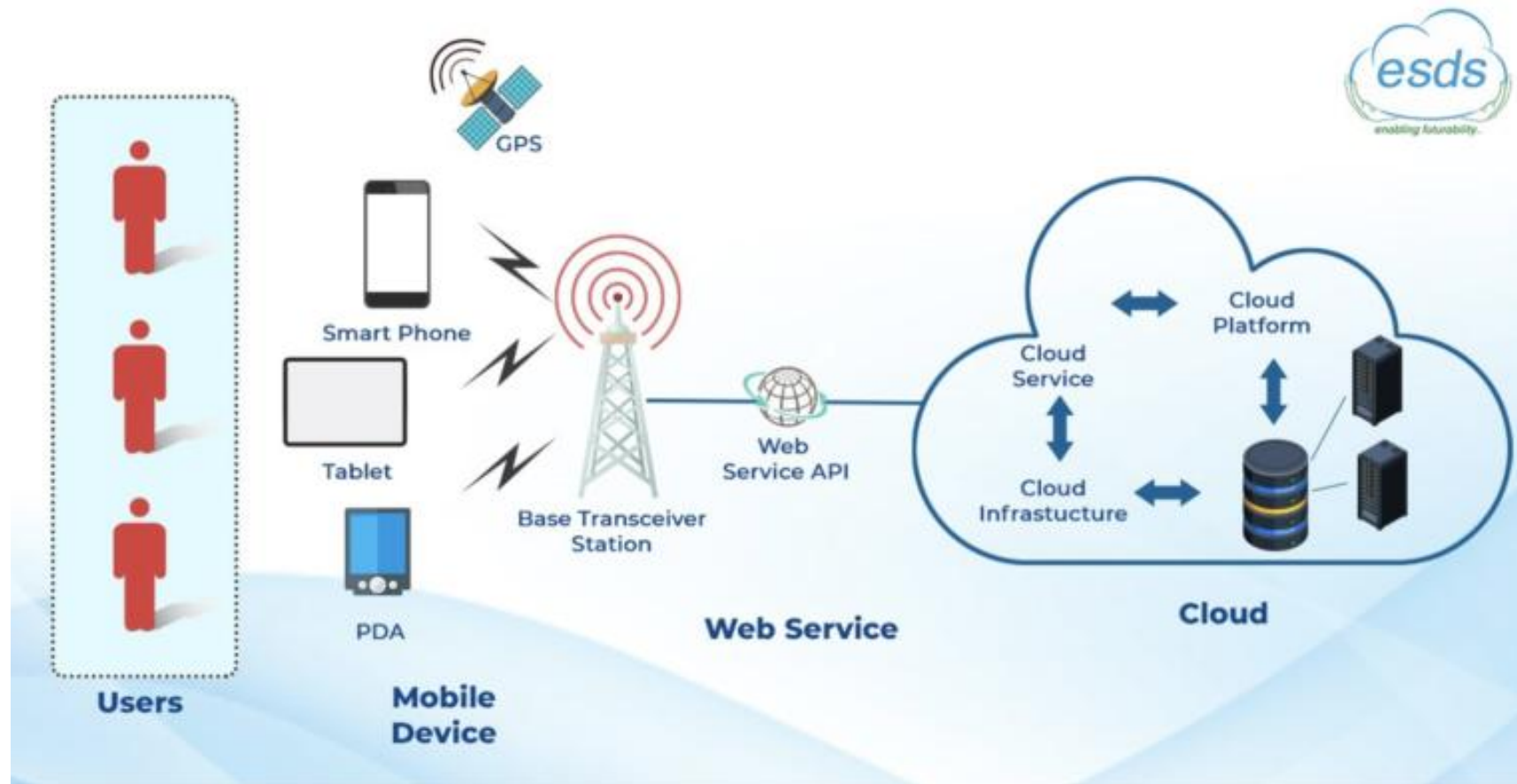
- ✓ Limited **battery power**
- ✓ Limited **CPU and memory**
- ✓ Limited **storage**
- ✓ Cannot handle **complex computations**
- ✓ Cloud computing solves these problems by providing:
- ✓ High processing power
- ✓ Large storage
- ✓ Scalability
- ✓ Anytime, anywhere access
- ✓ MCC = **Mobile Device + Cloud Power**

Basic Architecture of Mobile Cloud Computing

Mobile Device → Internet → Cloud Server → Internet → Mobile Device

Example: Face Recognition App

1. User **takes a photo** using a mobile phone
2. Image is sent to the cloud via the internet
3. Cloud server performs heavy **processing (AI algorithm)**
4. Result (**identified face**) is sent back to the mobile device
5. Mobile displays the result



Architectural Layers

- i. **Mobile Client Layer:** Individual devices (smartphones, tablets) running Client-Side Applications (CSA) that manage the interface and lightweight tasks.
- ii. **Mobile Network Layer:** Managed by Mobile Network Operators (MNOs), this includes **Base Stations and Access Points** that provide the wireless "air link" for data transmission.
- iii. **Cloud Backend Layer:** **Data centers where SaaS, PaaS, and IaaS services reside.** This layer handles heavy processing (computation offloading) and vast data storage.
- iv. **Operational Middleware:** Modern architectures often include **Cloudlets or Edge Nodes** that sit closer to the user to reduce latency for real-time applications.

Main Architectural Models

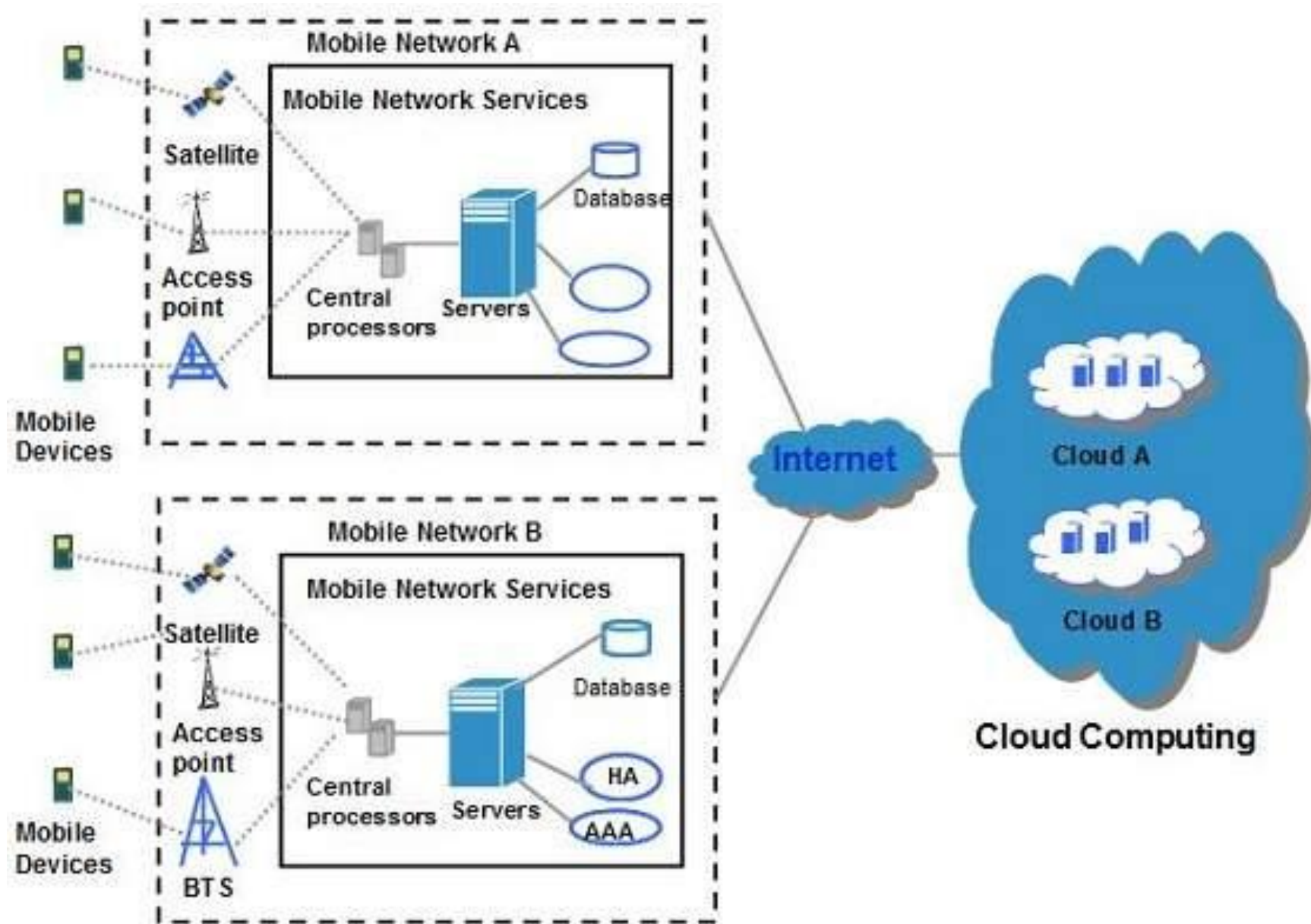
Non-Cloudlet Architecture: Mobile devices connect directly to a remote cloud, which can lead to higher latency.

Cloudlet Architecture: A middle layer (cloudlet) is added to provide immediate, local resource access, reducing latency and bandwidth consumption.

Core Functional Principles

Data Offloading: **Computationally intensive tasks** are transferred from the mobile device to the cloud, enhancing performance.

Virtual Machine (VM) Migration: Allows application mobility and state management between the device and the cloud.



Issues

Despite of having significant development in field of mobile cloud computing, still many issues remain unsorted such as:

- ✓ Emergency Efficient Transmission
- ✓ There should be a frequent transmission of information between cloud and the mobile devices.
- ✓ Architectural Issues
- ✓ Mobile cloud computing is required to make architectural neutral because of heterogeneous environment.

Live VM Migration

- ✓ It is challenging to migrate an application, which is resource-intensive to cloud and to execute it via Virtual Machine.
- ✓ Mobile Communication Congestion
- ✓ Due to continuous increase in demand for mobile cloud services, the workload to enable smooth communication between cloud and mobile devices has been increased.

Security and Privacy

- ✓ This is one of the major issues because mobile users share their personal information over the cloud.

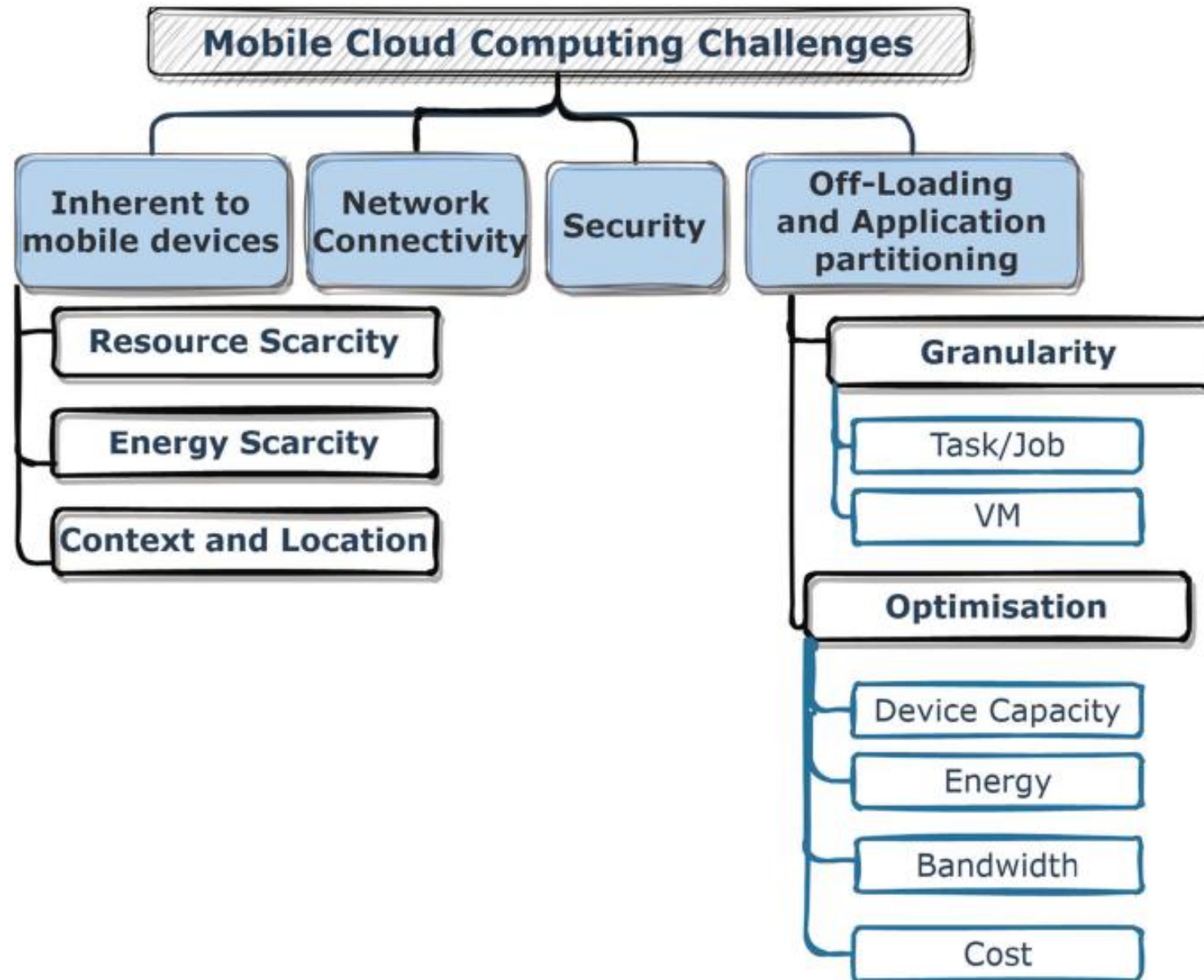
Feature	Explanation
Mobility	Access cloud services anytime, anywhere
Scalability	Cloud resources increase/decrease automatically
Energy Efficiency	Less battery consumption
Cost Effective	No need for powerful mobile hardware
Data Availability	Data accessible from any device

Applications of Mobile Cloud Computing

1. Education Google Classroom Online exams Cloud-based learning apps	2. Healthcare Remote patient monitoring Medical image processing	3. Business Mobile CRM Cloud-based ERP	4. Social Media Facebook, Instagram, Media stored and processed in cloud	5. Gaming Cloud gaming (e.g., game logic on cloud)
--	---	---	---	--

Aspect	Traditional Mobile	Mobile Cloud Computing
Processing	Mobile device	Cloud server
Storage	Limited	Large
Battery usage	High	Low
Performance	Limited	High
Scalability	No	Yes

Advantages	Challenges
✓ Better performance ✓ Longer battery life ✓ Reduced hardware cost ✓ High storage capacity ✓ Easy app updates	✓ Internet dependency ✓ Network latency ✓ Data security & privacy ✓ Bandwidth issues



Problem statement

Given Data: Assignment Details

- ✓ File size = **50 MB**
- ✓ Internet speed = **10 Mbps**
- ✓ Mobile CPU speed = **1.5 GHz**
- ✓ Cloud CPU speed = **15 GHz**

Processing Requirement

- I. Plagiarism detection requires **500 million CPU instructions**
- II. Analytics requires **300 million CPU instructions**
- III. Total instructions = **800 million**

Processing on Mobile Device (Without MCC)

Step 1: Calculate Execution Time on Mobile:

Execution Time=Instructions/CPU Speed
 $=\{800 \times 10^6\} / \{1.5 \times 10^9\} = 0.533$ seconds

Step 2: Energy Consumption on Mobile

Assume:

Mobile power consumption = 1.5 W

Energy=Power $\times$ Time= $1.5 \times 0.533 = 0.7995$ Joules

Result (Without MCC)

Time = 0.533 sec & Energy = 0.80 Joules

Processing Using Mobile Cloud Computing (With MCC)

Step 1: Upload Time to Cloud

Formula → Upload Time:

File Size/Bandwidth = $400/10 = 40$ seconds = 40 seconds [Convert units: 50 MB = **400 Mb**]

Step 2: Cloud Processing Time

Execution Time = $\{800 \times 10^6\} / \{15 \times 10^9\} = 0.053$ seconds

[Download Time] Result size = **1 MB = 8 Mb** = $8/10 = 0.8$ seconds

Total Time= $40+0.053+0.8 = 40.853$ seconds

Energy Consumption (MCC)

Assume:

Upload power = **0.6 W** & Download power = **0.4 W**

Energy_upload= $0.6 \times 40 = 24$ J

Energy_download= $0.4 \times 0.8 = 0.32$ J

Total Energy= 24.32 J

Parameter	Without MCC	With MCC
Processing Location	Mobile	Cloud
Execution Time	0.533 sec	40.853 sec
Energy Consumption	0.80 J	24.32 J
Battery Saving	✗	✗
Scalability	✗	✓
Suitable for AI Tasks	✗	✓

Another Example

- File size = **50 MB**
- Internet speed = **200 Mbps**
- Mobile CPU = **1.5 GHz**
- Cloud CPU = **15 GHz**
- Upload Power = **0.6 W**
- Download Power = **0.4 W**
- Processing = **800 million instructions**

Without MCC,

- Execution Time = 0.533 sec
- Energy = 0.80 J

With MCC,

$$\begin{aligned}\text{Total Time} &= \text{Upload Time} + \text{Cloud Processing} + \text{Download Time} \\ &= \mathbf{2+0.053+0.04 = 2.093 \text{ sec}}\end{aligned}$$

Energy Calculation

$$\text{Total Energy} = \mathbf{\text{Upload Energy} + \text{Download Energy} = 1.2+0.016=1.216\text{J}}$$

Code partitioning

What is Code Partitioning?

Code Partitioning is the process of **dividing an application's code into multiple parts** and deciding **which parts run on the mobile device and which parts run on the cloud.**

Code partitioning splits a mobile application so **that lightweight code runs on the phone** and **heavy code runs on the cloud.**

Why Code Partitioning is Needed in MCC

Mobile devices suffer from:

- a) Limited CPU and RAM
- b) Battery constraints
- c) Limited storage
- d) Cloud servers provide:
- e) High processing power
- f) Scalability
- g) Fast execution for heavy tasks

Example: Mobile Learning App

A mobile app performs:

- a) User login
- b) Video streaming
- c) Assignment evaluation (AI-based)

Application Module	Execution Location
User Interface	Mobile
Input validation	Mobile
Video decoding	Cloud
AI evaluation	Cloud
Result display	Mobile

Types of Code Partitioning

Static	Dynamic
Partitioning done at development time Fixed decision Example: AI module always runs in cloud ✓ Simple ✓ Not adaptable	Partitioning done at runtime Depends on: a) Network speed b) Battery level c) CPU load ✓ Efficient ✓ Adaptive ✓ Complex

Example

Given:

- 1) Local execution time = **2 seconds**
- 2) Cloud execution time = **0.2 seconds**
- 3) Data transfer time = **1.5 seconds**

Decision Rule:

Offload if: Cloud Time + Transfer Time < Local Time = $0.2 + 1.5 = 1.7 < 2$

Offloading beneficial.

Aspect	Code Partitioning	Computation Offloading
Meaning	Dividing code into parts	Moving whole task
Granularity	Fine-grained	Coarse-grained
Flexibility	High	Medium
Usage	Complex apps	Simple heavy tasks

Smart University App

- a) Student login & attendance → Mobile
- b) Face recognition for attendance → Cloud
- c) Result synchronization → Mobile

This is possible due to **code partitioning**.

Code Offloading in Mobile Cloud Computing

Code Offloading is a technique in which **part of an application's code or a complete computational task is transferred from a mobile device to a remote cloud server for execution**, and the result is sent back to the mobile device.

In MCC:

- ✓ Mobile executes **lightweight tasks**
- ✓ Cloud executes **heavy tasks**
- ✓ Results are returned to the mobile device

Typical Offloaded Tasks:

- ✓ Image processing
- ✓ Video rendering
- ✓ AI/ML model inference
- ✓ Data analytics

How Code Offloading Works

Example: Mobile Educational App

- i. Student uploads an image of a handwritten answer
- ii. Image-processing code is offloaded to the cloud
- iii. Cloud performs OCR and evaluation
- iv. Result is sent back to the mobile device
- v. Mobile displays the score

Dynamic & Static Offloading

Dynamic	Static
Static Code Offloading Offloading decision made at design time Fixed behavior Example: AI module always runs in cloud	Dynamic Code Offloading Offloading decision made at runtime Depends on: ✓ Network speed ✓ Battery level ✓ CPU usage Example: Offload only when Wi-Fi is available

Online Exam Evaluation System:

- a) Mobile captures answer scripts
- b) Evaluation code is offloaded to cloud
- c) Cloud processes thousands of students simultaneously
- d) Results returned instantly

This is **code offloading** in action

Aspect	Code Partitioning	Code Offloading
Definition	Dividing an application's code into multiple modules and deciding where each module should execute (mobile or cloud).	Transferring part of the application's code or a complete computational task from the mobile device to the cloud for execution.
Purpose	To determine the optimal execution location for different parts of the application.	To reduce the mobile device's processing load by executing heavy tasks remotely.
Focus	Planning/Decision – Which code runs where?	Execution – Sending the selected code to the cloud and running it.
When performed	Before or during execution (static or dynamic partitioning).	During application execution.
Granularity	Fine-grained (splits code into modules/functions).	Usually coarse-grained (offloads a module or task).
Output	A partitioned application (mobile part + cloud part).	Execution of the offloaded code in the cloud and returning the result.
Example	Login module → Mobile, AI module → Cloud, Result display → Mobile.	The AI module is sent to the cloud, executed there, and the result is returned to the phone.